

Aureon: A Production–Grade Reconciliation Rule Engine Design

1. Section 0 — Scope & Assumptions

1.1. Clarify the Scope

1.1.1. Target Institutions

The Aureon reconciliation engine is designed to serve a broad spectrum of financial institutions, each with unique operational complexities and regulatory requirements. The primary targets include **Portfolio Management Services (PMS)** , which manage individual client portfolios and require precise, client–level reconciliation. **Alternative Investment Funds (AIFs)** , including hedge funds and private equity, demand robust handling of complex derivatives, illiquid assets, and multi–currency portfolios, making reconciliation a critical risk management function. **Mutual funds**, with their high volume of daily transactions and focus on Net Asset Value (NAV) accuracy, are a key user group. The system must also cater to **custodians**, who are responsible for the safekeeping of assets and need to reconcile their records against fund managers and brokers, ensuring the safety and accuracy of client assets. **Wealth managers** and **family offices**, while often dealing with smaller volumes than large funds, manage highly complex and bespoke portfolios that include a wide range of asset classes, necessitating a flexible and comprehensive reconciliation system to maintain accurate books and records for high–net–worth individuals .

1.1.2. Asset Classes

The engine's scope encompasses a wide array of asset classes to ensure comprehensive coverage for modern investment portfolios. This includes **listed equities** and **Exchange–Traded Funds (ETFs)** , which form the bedrock of many portfolios and involve high transaction volumes. **Mutual funds**, both as a product offered by asset managers and as an investment held within other portfolios, are also a key focus. The system will handle **fixed–income securities**, such as government and corporate bonds, which introduce complexities like accrued interest and varying settlement cycles. **Foreign Exchange (FX)** trading, a fundamental component of global investing, requires specialized reconciliation logic to manage currency conversions and rate discrepancies . The engine will also cover **simple derivatives**, including futures and options, which present challenges related to margining, notional values, and expiration dates. Finally, **cash and cash equivalents** are included, as cash reconciliation is a fundamental

control for any financial institution, ensuring that internal ledger balances match external bank and custodian statements .

1.1.3. Geographies and Regulations

Aureon is designed to operate across major global financial markets, with specific considerations for regional regulations and settlement practices. The primary geographies include the **United States**, governed by the Securities and Exchange Commission (SEC) and the Depository Trust & Clearing Corporation (DTCC), which has recently moved to a **T+1 settlement cycle** for most securities. The **United Kingdom and European Union**, under the oversight of the Financial Conduct Authority (FCA) and the European Securities and Markets Authority (ESMA) respectively, have their own regulatory frameworks such as **UCITS** and **EMIR** that dictate operational standards. **India**, a rapidly growing market with its own unique ecosystem, operates under the Securities and Exchange Board of India (SEBI) and utilizes depositories like NSDL and CDSL, with a **T+1 settlement cycle**. The **Asia-Pacific (APAC)** region, including key financial centers like Singapore (regulated by MAS) and Hong Kong (regulated by the SFC), presents a diverse set of market practices and regulations. The engine will be built to accommodate these variations, particularly the critical differences in settlement cycles (T+1 vs. T+2) which directly impact the timing of cash and position reconciliations .

1.2. Explicit Assumptions

1.2.1. Data Availability

The design of the reconciliation engine assumes that certain core data fields are available from the various source systems, including internal books of record (IBOR), custodians, brokers, and fund administrators. For trade data, it is assumed that key fields such as **trade date, settlement date, ISIN or CUSIP identifier, quantity, price, currency, and trade side (buy/sell)** are present. For cash and position data, the system expects fields like **account number, currency, amount or quantity, and valuation date**. Corporate action data should include **event type, ex-date, record date, and payment date**. While the engine is designed to handle various file formats (PDF, CSV, XLSX, SWIFT), it is assumed that a preceding data ingestion and normalization layer will extract these critical fields and map them to a common internal schema. The availability of unique transaction identifiers (e.g., trade IDs from brokers) is also assumed to facilitate deterministic matching, although the system will be capable of performing fuzzy matching when such IDs are absent or inconsistent .

1.2.2. Reconciliation Frequency

The engine is designed to support both **daily batch and intraday reconciliation cycles**, with the specific frequency being configurable based on the institution's operational needs and the asset classes being managed. For most traditional asset managers and mutual funds, a **daily batch reconciliation** of trades, positions, and cash is the standard operational practice, typically performed after market close to ensure all day's activities are captured . This daily process is crucial for generating accurate NAVs and preparing for the next trading day. However, for more dynamic trading environments like hedge funds or firms with significant derivatives exposure, **intraday reconciliation** may be required to manage risk and ensure positions are accurately reflected in real-time. The system will be architected to handle these varying frequencies, allowing for the scheduling of different reconciliation jobs at different times of the day. The ability to perform daily reconciliations is considered a fundamental best practice to mitigate operational risk and ensure the integrity of portfolio data .

1.2.3. Settlement Cycles

The engine will be built to accommodate the different settlement cycles prevalent in global markets. The recent shift in the US to a **T+1 settlement cycle** for most securities means that trades executed today (T) will settle one business day later (T+1). This has significant implications for cash reconciliation, as the internal books will reflect a trade on the trade date, while the custodian's cash balance will only change on the settlement date . The engine must be able to differentiate between trade date and settlement date cash positions to avoid false breaks. In contrast, many markets in Europe and Asia still operate on a **T+2 cycle**. The system will therefore need configurable settlement cycle parameters for different markets and asset classes to ensure that timing differences are correctly identified and not flagged as genuine breaks. This is particularly important for managing liquidity and ensuring sufficient cash is available to meet settlement obligations .

1.2.4. Corporate Action Coverage

The reconciliation engine will be designed to handle the impact of **basic corporate actions** on portfolio positions and cash balances. This includes events such as **stock splits**, where the number of shares increases and the price decreases proportionally, and **reverse splits**, which have the opposite effect . **Bonus issues**, where shareholders receive additional shares for free, and **cash dividends**, which result in a cash inflow to the portfolio, are also within scope. The system will need to process these events

accurately to ensure that internal position records are adjusted correctly and that the resulting cash flows are reconciled with custodian statements. For example, when a stock split occurs, the engine must adjust the quantity of shares held and the average cost basis to reflect the new share structure, preventing a break in the position reconciliation. The system will also handle **simple rights issues**, where shareholders are given the right to purchase additional shares. More complex events like **mergers and demergers** will be handled at a high level, focusing on the resulting changes to positions and valuations .

2. Section 1 — Reconciliation Domains & Flows

2.1. Core Reconciliation Domains

2.1.1. Trade ↔ Cash

This domain involves the reconciliation of executed trades from brokers against the corresponding cash movements in bank or custodian accounts. The primary goal is to ensure that for every trade that has settled, the correct amount of cash has been debited or credited. This is a fundamental control to verify that the firm's financial records accurately reflect its trading activity and cash position. The process typically involves matching trades based on a unique identifier, or by using a combination of fields like ISIN, quantity, price, and settlement date. Discrepancies, or "breaks," in this domain can indicate issues such as incorrect trade booking, settlement failures, or errors in fee calculations . For example, if a buy trade for \$10,000 settles but the bank statement shows a debit of \$10,050, a break is generated to investigate the \$50 difference, which could be an unbooked brokerage fee.

2.1.2. Trade ↔ Custodian / Contract Notes

This reconciliation domain focuses on verifying that the trades executed by the firm match the information provided by the custodian or broker in the form of contract notes or settlement confirmations. This process ensures that the trade details, such as security, quantity, price, and settlement date, are consistent between the firm's internal records and the external counterparty's records. It is a critical step in confirming that the firm is receiving the correct securities and paying the correct amount for them. Breaks in this area can arise from timing differences, where a trade is booked internally on the trade date but appears on the custodian's report on the settlement date, or from genuine errors in trade execution or reporting . For instance, a mismatch in the price of

a trade between the internal system and the contract note would trigger a break that needs to be resolved with the broker.

2.1.3. Positions ↔ Custodian

Position reconciliation is the process of comparing the firm's internal records of its security holdings (its **Investment Book of Record**, or IBOR) with the statements provided by its custodians (the **Custodian Book of Record**, or CBOR) . This is a daily, best-practice control to ensure the integrity of the assets held on behalf of the firm and its clients. The reconciliation involves matching the quantity of each security held, as well as the market value, which is derived from the price and quantity. Breaks can occur due to a variety of reasons, including unsettled trades, corporate actions that have not been processed correctly on one side, or errors in data entry . For example, if the internal system shows a holding of 1,000 shares of a company, but the custodian's statement shows 950 shares, a break is generated to investigate the 50-share difference, which could be due to a pending sale that has not yet settled.

2.1.4. NAV & AUM Checks

This domain involves the reconciliation of the fund's **Net Asset Value (NAV)** and **Assets Under Management (AUM)** as calculated internally against the values provided by an external fund administrator or accounting agent. The NAV is a critical metric for mutual funds and other pooled investment vehicles, as it determines the price at which investors buy and sell shares in the fund. AUM is a key performance indicator for asset managers. Discrepancies in these values can have significant financial and reputational consequences. Breaks in NAV reconciliation are often caused by valuation errors, such as using an incorrect price for a security, or by misallocating income and expenses to the wrong fund or share class . For example, if the internal NAV calculation for a mutual fund is \$10.05 per share, but the fund administrator calculates it as \$10.03, a break is generated to identify the source of the 2-cent difference.

2.1.5. Fees & Charges

This reconciliation domain focuses on ensuring that all fees and charges incurred by the fund, such as **management fees, brokerage commissions, and custody fees**, are accurately calculated, booked, and paid. It involves comparing the fees calculated internally with the invoices and statements received from service providers like investment managers, brokers, and custodians. Breaks can arise from discrepancies in the fee calculation methodology, incorrect application of fee rates, or timing

differences in when the fees are recognized . For example, if a custodian's statement includes a safekeeping fee of \$500 that has not yet been accrued in the internal books, a break is generated to ensure the fee is properly recorded and paid.

2.1.6. Corporate Actions Impact

This domain deals with the reconciliation of corporate actions, such as **dividends, stock splits, and mergers**, to ensure they are processed accurately and consistently across all systems. The process involves verifying the details of the corporate action event, such as the ex-date, record date, and payment date, and ensuring that the resulting changes to positions and cash flows are correctly reflected in both the internal records and the custodian's statements . Breaks can occur if the corporate action is not processed on time, if the entitlement is calculated incorrectly (e.g., due to a mistake in the tax rate applied to a dividend), or if there is a timing difference between when the event is booked internally and when it is reflected by the custodian . For example, if a dividend is paid on a stock, but the amount received from the custodian differs from the amount expected due to a change in withholding tax, a break is generated.

2.1.7. FX & Multi-Currency Valuation

This reconciliation domain is crucial for funds that invest in foreign securities or trade in multiple currencies. It involves reconciling the **foreign exchange (FX) rates** used to convert transaction amounts and valuations from the local currency to the fund's base currency. The process also includes reconciling the cash balances held in different currency accounts. Breaks can be caused by using different FX rate sources (e.g., the fund uses a Bloomberg rate while the custodian uses its own internal rate), timing differences in when the rates are applied, or rounding errors in the conversion process . For example, if a fund converts €1 million to USD at a rate of 1.10, but the custodian uses a rate of 1.11, a break of \$10,000 will arise that needs to be investigated .

2.1.8. Suspense & Break Management

This is a catch-all domain that deals with the management of all reconciliation breaks and exceptions that cannot be automatically resolved. It involves categorizing breaks by type (e.g., data, timing, economic), prioritizing them based on their potential impact (e.g., on NAV, regulatory reporting), and routing them to the appropriate personnel for investigation and resolution . The system will track the aging of breaks to ensure they are resolved in a timely manner and to identify recurring issues that may indicate a

systemic problem. For example, a break that has been unresolved for more than five business days might be escalated to a senior operations manager, while a break that impacts the fund's NAV would be flagged as critical and require immediate attention . This domain is essential for maintaining an audit trail of all discrepancies and their resolutions, which is a key requirement for regulatory compliance .

2.2. Domain Specifications

2.2.1. What is Being Reconciled

In the **Trade ↔ Cash** domain, the primary focus is on matching the financial impact of settled trades. This involves comparing the trade amount (quantity * price +/- fees) from the internal trade blotter against the corresponding cash debit or credit on the bank or custodian statement. The reconciliation must account for the trade's currency and the settlement date to ensure an accurate match. In the **Positions ↔ Custodian** domain, the core task is to verify the ownership and valuation of assets. This means comparing the quantity of each security held in the internal Investment Book of Record (IBOR) with the quantity reported by the custodian. Additionally, the market value of these positions, calculated using the price and quantity, must also be reconciled to ensure accurate portfolio valuation and NAV calculation . The **NAV & AUM Checks** domain involves a high-level reconciliation of the fund's overall value. This means comparing the total NAV and AUM figures calculated internally with those provided by an external fund administrator, ensuring that all underlying components, including positions, cash, and accruals, are in agreement.

2.2.2. Typical File Types / Data Sources

The reconciliation engine will ingest data from a variety of sources and in multiple formats. For trade data, sources will include Order Management Systems (OMS), Execution Management Systems (EMS), and direct feeds from brokers. This data may be provided in formats such as CSV, Excel (XLSX), or proprietary flat files. Cash and position data will typically come from custodians and fund administrators, often delivered as **SWIFT messages** (e.g., MT940 for cash balances, MT535 for position statements) or through secure file transfer protocols (SFTP) in CSV or XML formats . Corporate action data may be sourced from specialized data providers like Bloomberg, Refinitiv, or directly from custodians. The engine's data ingestion layer must be capable of parsing these diverse formats and normalizing them into a common, structured data model for comparison. This flexibility is crucial for integrating with the varied technological landscapes of different financial institutions and their service providers .

2.2.3. What “Good” Looks Like

A "good" or successful reconciliation is one where all records match perfectly between the internal and external sources, with no outstanding breaks or exceptions. In the **Trade ↔ Cash** domain, this means that for every settled trade, there is a corresponding and identical cash entry in the bank or custodian account, with any differences being fully explained by known and booked fees or charges. For **Positions ↔ Custodian**, a good reconciliation results in a perfect match for every security's quantity and market value between the internal IBOR and the custodian's statement . In the **NAV & AUM Checks** domain, a successful outcome is when the internally calculated NAV and AUM figures are identical to those produced by the fund administrator, indicating that all underlying data and calculations are in sync. Achieving a "good" state requires not only accurate data but also robust matching logic, appropriate tolerance levels, and efficient exception management processes to quickly identify and resolve any discrepancies .

2.2.4. What a “Break” Usually Means

A "break" in reconciliation signifies a discrepancy between two sets of records that should, in theory, be identical. In the **Trade ↔ Cash** domain, a break typically means that the cash impact of a trade does not match the expected amount, which could be due to a settlement failure, an unbooked fee, or an error in the trade's price or quantity . In **Positions ↔ Custodian**, a break indicates a difference in the quantity or value of a security holding. This could be caused by an unsettled trade, an incorrectly processed corporate action, or a data entry error . For **NAV & AUM Checks**, a break signifies a difference in the fund's overall valuation, which could stem from a pricing error, a missing accrual, or a corporate action that has not been applied consistently. In essence, a break is a signal that something in the post-trade process has gone wrong and requires investigation to determine the root cause and take corrective action .

3. Section 2 — Detailed Rule Catalog (The Heart)

3.1. Trade ↔ Cash Reconciliation Rules

3.1.1. Deterministic Rules

- **id:** TRAC_001
domain: Trade ↔ Cash
name: Unique Reference Match
type: DETERMINISTIC
description: This rule performs an exact match on a unique transaction identifier,

such as a trade reference number or a payment reference. This is the most fundamental and reliable matching criterion, as a unique ID should correspond to a single transaction on both the trade and cash sides. A successful match on this field provides a high degree of confidence that the two records represent the same event.

inputs:

- `trade_reference` : The unique identifier for the trade from the internal system.
- `cash_reference` : The reference number from the bank or custodian statement.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM trades t  
INNER JOIN cash c ON t.trade_reference = c.cash_reference
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : This is a core deterministic rule that forms the basis of the matching process.

severity:

- `level` : CRITICAL
- `impact_if_broken` : A failure to match on a unique reference indicates a fundamental data integrity issue, potentially leading to unmatched trades and incorrect cash balances.

example_breaks:

- "Example 1: A trade is executed with reference 'TRD12345', but the corresponding bank statement entry shows a reference 'PAY67890'. This could

be due to a bank-specific formatting of the reference or a mismatch in the data feed."

- "Example 2: The trade reference is missing from the bank statement, making a direct match impossible."

example_fix:

- "The operations team would need to investigate the source of the reference mismatch. This might involve checking with the broker or bank to understand their reference generation logic or correcting the data mapping in the ingestion layer."

- **id:** TRAC_002

domain: Trade ↔ Cash

name: Amount Exact Match

type: DETERMINISTIC

description: This rule performs an exact match on the transaction amount. For a trade, this would be the net settlement amount (gross amount plus/minus fees and taxes). For a cash entry, this would be the debit or credit amount. This rule is used in conjunction with other criteria, such as date and currency, to identify matching pairs.

inputs:

- `trade_net_amount` : The net settlement amount of the trade.
- `cash_amount` : The amount of the cash entry.
- `trade_currency` : The currency of the trade.
- `cash_currency` : The currency of the cash entry.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM trades t
INNER JOIN cash c ON t.trade_net_amount = c.cash_amount
WHERE t.trade_currency = c.cash_currency
```

suggested_tolerances:

- `amount_abs` : 0

- `amount_pct` : 0%
- `date_offset_days` : 0
- deterministic_or_ai:**
- `should_live_in` : "RULE_ENGINE"
- `notes` : A fundamental rule for ensuring the financial integrity of the reconciliation.
- severity:**
- `level` : HIGH
- `impact_if_broken` : An amount mismatch can indicate a booking error, an unrecorded fee, or a partial settlement, all of which have a direct financial impact.

example_breaks:

- "Example 1: A trade for \$10,000 is booked, but the bank statement shows a debit of \$10,025. The \$25 difference could be a wire transfer fee that was not recorded in the trade details."
- "Example 2: A trade is booked in GBP, but the cash movement is in USD, requiring an FX conversion to be performed for matching."

example_fix:

- "The analyst would first check for any associated fees or taxes that might explain the difference. If no such items are found, the trade details and bank statement would need to be verified with the broker and bank, respectively."

- **id:** TRAC_003

domain: Trade ↔ Cash

name: Currency Match

type: DETERMINISTIC

description: This rule ensures that the currency of the trade matches the currency of the corresponding cash entry. This is a critical check to prevent mismatches that could arise from cross-currency transactions or incorrect data entry. A currency mismatch will prevent a match from being made, even if the amounts are numerically identical.

inputs:

- `trade_currency` : The currency of the trade.

- `cash_currency` : The currency of the cash entry.
`logic_pseudocode_sql`: |

sql

📄 复制

```
SELECT *  
FROM trades t  
INNER JOIN cash c ON t.trade_currency = c.cash_currency
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A simple but essential rule for data quality and preventing incorrect matches.

severity:

- `level` : HIGH
- `impact_if_broken` : A currency mismatch can lead to significant errors in cash and NAV calculations, especially in volatile FX markets.

example_breaks:

- "Example 1: A trade is executed in EUR, but the cash is received in a USD account, indicating a cross-currency settlement that needs to be handled by FX-specific rules."
- "Example 2: A data entry error results in a trade being booked with the wrong currency code (e.g., 'GBP' instead of 'USD')."

example_fix:

- "The analyst would need to identify the correct currency for the transaction and correct the data in the relevant system. This may also involve checking the settlement instructions for the trade."

- **id:** TRAC_004

domain: Trade ↔ Cash

name: ISIN/Security Match

type: DETERMINISTIC

description: This rule matches trades and cash entries based on the security identifier, such as an ISIN or CUSIP. This is particularly important for trades that result in a cash flow, such as a dividend payment or a bond maturity. The rule ensures that the cash received is correctly attributed to the specific security that generated it.

inputs:

- `trade_isin` : The ISIN of the security in the trade.
 - `cash_isin` : The ISIN of the security associated with the cash entry (if available).
- logic_pseudocode_sql:** |

sql

📄 复制

```
SELECT *  
FROM trades t  
INNER JOIN cash c ON t.trade_isin = c.cash_isin
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : Essential for correctly attributing cash flows to the underlying securities.

severity:

- `level` : MEDIUM
- `impact_if_broken` : A mismatch can lead to incorrect attribution of income or proceeds, affecting performance reporting and tax calculations.

example_breaks:

- "Example 1: A dividend is received for a security, but the ISIN on the bank statement is a temporary ISIN used during a corporate action, which does not match the ISIN in the internal system."

- "Example 2: A cash entry for a bond maturity is missing the ISIN, making it impossible to match to the specific bond position."

example_fix:

- "The analyst would need to use other information, such as the amount and date, to identify the correct security. This may also involve creating a mapping for temporary or alias ISINs."

- **id:** TRAC_005

domain: Trade ↔ Cash

name: Settlement Date Match

type: DETERMINISTIC

description: This rule matches trades and cash entries based on the expected settlement date. For a trade, this is the date on which the cash and securities are expected to be exchanged. For a cash entry, this is the value date of the transaction. This rule is crucial for identifying timing differences, where a trade has been booked but has not yet settled.

inputs:

- `trade_settlement_date` : The expected settlement date of the trade.

- `cash_value_date` : The value date of the cash entry.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM trades t
INNER JOIN cash c ON t.trade_settlement_date = c.cash_value_date
```

suggested_tolerances:

- `amount_abs` : 0

- `amount_pct` : 0%

- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A key rule for identifying timing differences and managing settlement risk.
severity:
- `level` : MEDIUM
- `impact_if_broken` : A date mismatch can indicate a settlement failure or a timing difference that needs to be monitored.
example_breaks:
- "Example 1: A trade is booked with a settlement date of 2024-12-05, but the cash movement occurs on 2024-12-06, possibly due to a settlement delay."
- "Example 2: A trade is booked on a Friday with a T+2 settlement cycle, but the cash movement occurs on the following Tuesday, which is the correct settlement date."
- example_fix:**
- "The analyst would check the settlement calendar to confirm the expected settlement date. If the trade has failed to settle, they would need to follow up with the broker to resolve the issue."

3.1.2. Tolerance-Based Rules

- **id:** TRAC_006
domain: Trade ↔ Cash
name: Amount Tolerance Match
type: TOLERANCE
description: This rule allows for a match between a trade and a cash entry even if their amounts are not identical, provided the difference falls within a predefined tolerance. This is essential for handling minor discrepancies caused by rounding differences, small, unitemized bank fees, or slight variations in how interest is calculated. The tolerance can be defined as an absolute amount (e.g., +/- \$0.01) or a percentage of the transaction amount (e.g., +/- 0.1%). This rule significantly reduces the number of manual investigations required for trivial differences.
inputs:
- `trade_net_amount` : The net cash amount of the trade.
- `cash_amount` : The cash amount from the bank statement.

- `tolerance_abs` : The absolute tolerance allowed.
 - `tolerance_pct` : The percentage tolerance allowed.
- logic_pseudocode_sql:** |

sql

📄 复制

```
SELECT *
FROM trades t
JOIN cash c
  ON ABS(t.trade_net_amount - c.cash_amount) <= @tolerance_abs
 AND ABS((t.trade_net_amount - c.cash_amount) /
t.trade_net_amount) * 100 <= @tolerance_pct
```

suggested_tolerances:

- `amount_abs` : 0.01
- `amount_pct` : 0.1%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : While it uses a tolerance, the logic is still rule-based and can be implemented in SQL.

severity:

- `level` : MEDIUM
- `impact_if_broken` : A small, persistent difference could indicate a systematic issue with fee calculations or rounding rules that needs to be addressed.

example_breaks:

- "Example 1: A trade for \$1,000.00 is matched with a cash entry for \$999.98, a difference of \$0.02, which is within the tolerance."
- "Example 2: A trade for \$1,000.00 is matched with a cash entry for \$995.00, a difference of \$5.00, which is outside the tolerance and flagged as a break."

example_fix:

- "If the difference is within tolerance, the analyst may approve the match. If it's a recurring small difference, they might investigate the cause (e.g., a bank fee) and

create a new rule to account for it automatically in the future."

- **id:** TRAC_007

domain: Trade ↔ Cash

name: Date Tolerance Match

type: TOLERANCE

description: This rule allows for matching when the value date of the cash movement is one or two days different from the expected value date of the trade. This can happen due to weekends, holidays, or slight delays in the settlement process.

inputs:

- `trade_settlement_date` : The expected settlement date of the trade.
- `cash_value_date` : The value date of the cash entry.
- `date_tolerance_days` : The number of days allowed for a date difference.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM trades t
JOIN cash c
  ON ABS(DATEDIFF(day, t.trade_settlement_date,
c.cash_value_date)) <= @date_tolerance_days
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 2

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A rule-based tolerance for handling common settlement timing issues.

severity:

- `level` : MEDIUM

- `impact_if_broken` : A date mismatch outside of tolerance can indicate a settlement failure or a significant delay that needs to be investigated.

example_breaks:

- "Example 1: A trade with a settlement date of 2024-12-05 (a Friday) is matched with a cash transaction on 2024-12-09 (the following Monday), which is within the 2-day offset tolerance."
- "Example 2: A trade with a settlement date of 2024-12-05 is not matched with any cash transaction within a 2-day window."

example_fix:

- "The analyst would check the settlement calendar to see if the delay is due to a holiday or weekend. If so, they would expect the settlement to occur on the next business day. If the delay is longer than expected, they would contact the broker to inquire about the status of the settlement."

3.1.3. Data Quality Rules

- **id:** TRAC_008

domain: Trade ↔ Cash

name: Missing Trade Reference

type: DATA_QUALITY

description: This rule flags any trade or cash entry that is missing a unique reference number. A missing reference makes it difficult to perform a deterministic match and increases the risk of incorrect or ambiguous matching. This rule helps to identify data quality issues in the source systems.

inputs:

- `trade_reference` : The unique identifier for the trade.
- `cash_reference` : The reference number from the bank or custodian statement.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM trades t
WHERE t.trade_reference IS NULL OR t.trade_reference = ''
UNION
SELECT *
```

```
FROM cash c
WHERE c.cash_reference IS NULL OR c.cash_reference = ''
```

suggested_tolerances:

- amount_abs : N/A
- amount_pct : N/A
- date_offset_days : N/A

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A simple data quality check that can be implemented in SQL.

severity:

- level : MEDIUM
- `impact\_if\_broken`: Missing references can lead to increased manual effort and a higher risk of incorrect matching.

example_breaks:

- "Example 1: A trade record is received from the OMS with a null value in the trade reference field."
- "Example 2: A cash entry from the bank statement has an empty reference field."

example_fix:

- "The analyst would need to investigate the source of the data and work with the data provider to ensure that reference numbers are consistently populated. In the short term, they may need to use other fields to match the item manually."

• **id:** TRAC_009

domain: Trade ↔ Cash

name: Invalid Currency Code

type: DATA_QUALITY

description: This rule checks for invalid or unrecognized currency codes in the trade and cash data. This is a critical data quality check, as an incorrect currency code can lead to significant errors in reconciliation and valuation. The rule will compare the currency codes against a standard list of valid ISO 4217 currency codes.

inputs:

- trade_currency : The currency of the trade.

- `cash_currency` : The currency of the cash entry.

`logic_pseudocode_sql`: |

sql

📄 复制

```
SELECT *
FROM trades t
WHERE t.trade_currency NOT IN (SELECT currency_code FROM
valid_currencies)
UNION
SELECT *
FROM cash c
WHERE c.cash_currency NOT IN (SELECT currency_code FROM
valid_currencies)
```

suggested_tolerances:

- `amount_abs` : N/A
- `amount_pct` : N/A
- `date_offset_days` : N/A

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A straightforward data quality check against a reference table.

severity:

- `level` : HIGH
- `impact_if_broken` : Invalid currency codes can lead to incorrect FX conversions and valuation errors.

example_breaks:

- "Example 1: A trade is booked with the currency code 'US' instead of the standard 'USD'."
- "Example 2: A cash entry has a typo in the currency code, such as 'EUP' instead of 'EUR'."

example_fix:

- "The analyst would correct the currency code in the source data and re-run the reconciliation. They would also work with the data provider to prevent such

errors in the future."

3.1.4. Edge-Case Rules

- **id:** TRAC_010

domain: Trade ↔ Cash

name: Partial Fill Detection

type: EDGE_CASE

description: This rule is designed to detect when a trade has been partially filled, meaning that only a portion of the original order quantity has been executed. This is common for large orders or in illiquid markets. The rule will match the executed quantity and amount from the trade confirmation with the corresponding cash movement, while the remaining unfilled portion of the order will be tracked separately.

inputs:

- `trade_quantity` : The original quantity of the trade order.
- `executed_quantity` : The quantity that was actually executed.
- `trade_net_amount` : The net amount for the executed quantity.
- `cash_amount` : The cash amount from the bank statement.

logic_pseudocode_sql: |

Python

📄 复制

```
# Pseudocode for logic
for trade in trades:
    if trade.executed_quantity < trade.trade_quantity:
        # This is a partial fill
        expected_amount = (trade.executed_quantity /
trade.trade_quantity) * trade.trade_net_amount
        # Now match expected_amount with cash entries
```

suggested_tolerances:

- `amount_abs` : 0.01
- `amount_pct` : 0.1%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : The logic is deterministic but requires more complex calculations than a simple match.

severity:

- `level` : MEDIUM
- ``impact_if_broken**`: Incorrect handling of partial fills can lead to inaccurate position and cash reporting.

example_breaks:

- "Example 1: An order for 10,000 shares is placed, but only 5,000 shares are executed. The cash reconciliation must match the amount for the 5,000 shares, while the remaining 5,000 shares are still an open order."
- "Example 2: A partial fill is not correctly identified, leading to a break because the cash amount does not match the full order amount."

example_fix:

- "The analyst would verify the trade confirmation to confirm the executed quantity and amount. They would then ensure that the reconciliation system is configured to correctly handle partial fills and track the remaining open orders."

- **id:** TRAC_011

domain: Trade ↔ Cash

name: Trade Cancellation and Correction

type: EDGE_CASE

description: This rule handles trades that have been cancelled or corrected after they were initially booked. A cancellation will result in a reversal of the original trade's cash impact, while a correction will involve a new trade with the correct details. The rule must be able to identify these events and ensure that the net cash impact is correctly reflected in the reconciliation.

inputs:

- `trade_status` : The status of the trade (e.g., 'CANCELLED', 'CORRECTED').
- `original_trade_id` : The ID of the original trade that was cancelled or corrected.
- `cash_narrative` : The description of the cash entry, which may indicate a reversal.

logic_pseudocode_sql: |

```
# Pseudocode for logic
for trade in trades:
    if trade.trade_status == 'CANCELLED':
        # Look for a corresponding cash entry that reverses the
        # original trade

        find_cash_entry_with_opposite_amount_and_same_reference(trade)
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : The logic is deterministic but requires careful handling of trade statuses and cash narratives.

severity:

- `level` : HIGH
- `impact_if_broken` : Failure to correctly process cancellations and corrections can lead to significant errors in cash and position reporting.

example_breaks:

- "Example 1: A trade is cancelled, but the reversal is not correctly reflected in the cash ledger, leading to a duplicate cash entry."
- "Example 2: A trade correction is not properly linked to the original trade, resulting in two separate, unmatched trades."

example_fix:

- "The analyst would verify the cancellation or correction notice from the broker and ensure that the internal records are updated correctly. They would also check that the corresponding cash reversal has been processed by the custodian."

3.2. Positions ↔ Custodian Reconciliation Rules

3.2.1. Deterministic Rules

- **id:** POS_001

domain: Positions ↔ Custodian

name: Exact Quantity Match

type: DETERMINISTIC

description: This is the most fundamental rule for position reconciliation. It performs an exact match of the quantity of each security held in the internal system against the quantity reported by the custodian. This rule is applied on a security-by-security basis and is the primary control for ensuring the accuracy of the fund's holdings.

inputs:

- `internal_isin` : The ISIN of the security in the internal holdings file.
- `internal_quantity` : The quantity of the security held internally.
- `custodian_isin` : The ISIN of the security in the custodian's statement.
- `custodian_quantity` : The quantity of the security held by the custodian.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM internal_holdings ih
FULL OUTER JOIN custodian_holdings ch
  ON ih.internal_isin = ch.custodian_isin
WHERE ih.internal_quantity != ch.custodian_quantity
      OR (ih.internal_isin IS NULL AND ch.custodian_isin IS NOT NULL)
      OR (ih.internal_isin IS NOT NULL AND ch.custodian_isin IS NULL)
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"

- `notes` : This is a core deterministic rule that requires an exact match.

severity:

- `level` : CRITICAL

- `impact_if_broken` : A quantity mismatch indicates a serious discrepancy in the fund's holdings, which can lead to incorrect NAV, risk reporting, and client statements.

example_breaks:

- "Example 1: The internal system shows a holding of 1,000 shares of ISIN US1234567890, but the custodian's statement shows 950 shares."
- "Example 2: The custodian's statement includes a security that is not present in the internal holdings file."

example_fix:

- "The analyst would investigate the source of the discrepancy. This could be due to a trade that has not yet settled, a corporate action that has not been processed, or a data entry error. The fix would involve correcting the underlying cause of the break."

- **id:** POS_002

domain: Positions ↔ Custodian

name: ISIN Match

type: DETERMINISTIC

description: This rule ensures that the security identifier (ISIN) in the internal holdings file matches the ISIN in the custodian's statement. This is a critical check to ensure that the reconciliation is comparing the same securities. Any mismatch in the ISIN will prevent a match from being made and will be flagged as a data quality issue.

inputs:

- `internal_isin` : The ISIN of the security in the internal holdings file.
- `custodian_isin` : The ISIN of the security in the custodian's statement.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM internal_holdings ih
```

```
JOIN custodian_holdings ch
  ON ih.internal_isin = ch.custodian_isin
WHERE ih.internal_isin != ch.custodian_isin
```

suggested_tolerances:

- amount_abs : 0
- amount_pct : 0%
- date_offset_days : 0

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A fundamental deterministic rule for data integrity.

severity:

- level : HIGH
- impact_if_broken : An ISIN mismatch can lead to incorrect valuation and risk calculations, as the wrong security is being considered.

example_breaks:

- "Example 1: The internal system uses the ISIN 'US0378331005' for Apple Inc., but the custodian's statement uses a different identifier, such as a CUSIP or a local ticker."
- "Example 2: A data mapping error results in an incorrect ISIN being populated in the internal holdings file."

example_fix:

- "The analyst would verify the correct ISIN for the security and correct the data in the relevant system. This may also involve creating a mapping between different identifier types."

3.2.2. Tolerance-Based Rules

- id: POS_003

domain: Positions ↔ Custodian

name: Market Value Tolerance Match

type: TOLERANCE

description: This rule allows for a match in the market value of a position even if

there is a small difference, provided the difference is within a predefined tolerance. This is often necessary to account for minor discrepancies in the pricing sources used by the internal system and the custodian, or for rounding differences.

inputs:

- `internal_market_value` : The market value of the position from the internal system.
- `custodian_market_value` : The market value of the position from the custodian's statement.
- `tolerance_abs` : The absolute tolerance allowed.
- `tolerance_pct` : The percentage tolerance allowed.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM internal_holdings ih
JOIN custodian_holdings ch
  ON ih.internal_isin = ch.custodian_isin
WHERE ABS(ih.internal_market_value - ch.custodian_market_value) <=
@tolerance_abs
  OR ABS((ih.internal_market_value - ch.custodian_market_value) /
ih.internal_market_value) * 100 <= @tolerance_pct
```

suggested_tolerances:

- `amount_abs` : 10.00
- `amount_pct` : 0.1%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A rule-based tolerance for handling minor valuation differences.

severity:

- `level` : MEDIUM

- `impact_if_broken` : A persistent market value difference could indicate a systematic issue with pricing data that needs to be investigated.

example_breaks:

- "Example 1: The internal market value for a position is \$100,050, while the custodian's market value is \$100,040, a difference of \$10, which is within the tolerance."
- "Example 2: The internal market value for a position is \$100,050, while the custodian's market value is \$99,950, a difference of \$100, which is outside the tolerance."

example_fix:

- "If the difference is within tolerance, the analyst may approve the match. If it's a recurring issue, they might investigate the pricing sources and work to align them."

3.2.3. Data Quality Rules

- **id:** POS_004

domain: Positions ↔ Custodian

name: Missing ISIN

type: DATA_QUALITY

description: This rule flags any position in the internal or custodian data that is missing a security identifier (ISIN). A missing ISIN makes it impossible to perform a deterministic match and can lead to serious errors in reconciliation and reporting.

inputs:

- `internal_isin` : The ISIN of the security in the internal holdings file.
- `custodian_isin` : The ISIN of the security in the custodian's statement.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM internal_holdings ih
WHERE ih.internal_isin IS NULL OR ih.internal_isin = ''
UNION
SELECT *
```

```
FROM custodian_holdings ch
WHERE ch.custodian_isin IS NULL OR ch.custodian_isin = ''
```

suggested_tolerances:

- amount_abs : N/A
- amount_pct : N/A
- date_offset_days : N/A

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A simple but critical data quality check.

severity:

- level : HIGH
- impact_if_broken : A missing ISIN prevents the position from being correctly identified and reconciled.

example_breaks:

- "Example 1: A new security is added to the portfolio, but the ISIN is not yet available in the internal system."
- "Example 2: The custodian's statement for a particular security is missing the ISIN."

example_fix:

- "The analyst would need to identify the correct ISIN for the security and update the data in the relevant system. This may involve checking with the data provider or using a security master database."

3.2.4. Edge-Case Rules

- **id:** POS_005

domain: Positions ↔ Custodian

name: Unsettled Trade Adjustment

type: EDGE_CASE

description: This rule is designed to explain position breaks that are due to unsettled trades. A trade that has been executed but has not yet settled will appear in the internal position file but not in the custodian's statement (for a buy) or vice

versa (for a sell). This rule will identify such trades and provide a clear explanation for the break, preventing it from being flagged as a genuine discrepancy.

inputs:

- `internal_isin` : The ISIN of the security in the internal holdings file.
- `internal_quantity` : The quantity of the security held internally.
- `custodian_isin` : The ISIN of the security in the custodian's statement.
- `custodian_quantity` : The quantity of the security held by the custodian.
- `trade_isin` : The ISIN of the security in the trade file.
- `trade_quantity` : The quantity of the trade.
- `trade_settlement_date` : The settlement date of the trade.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM position_breaks pb
JOIN trades t
    ON pb.isin = t.trade_isin
WHERE t.trade_settlement_date > @current_date
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : The logic is deterministic but requires joining with the trade file.

severity:

- `level` : LOW
- `impact_if_broken` : Unsettled trades are a normal part of the settlement process and should not be treated as a genuine break.

example_breaks:

- "Example 1: A buy trade for 1,000 shares is executed on T, but the settlement date is T+2. On T+1, the internal system shows 1,000 shares, but the custodian's statement shows 0 shares."
- "Example 2: A sell trade for 500 shares is executed on T, but the settlement date is T+2. On T+1, the internal system shows 0 shares, but the custodian's statement still shows 500 shares."

example_fix:

- "The analyst would identify the unsettled trade as the cause of the break and would monitor the trade until it settles on the expected date. No manual intervention is typically required."

3.3. NAV & Valuation Checks Rules

3.3.1. Deterministic Rules

- **id:** NAV_001

domain: NAV & Valuation Checks

name: Exact NAV Match

type: DETERMINISTIC

description: This rule performs an exact match of the Net Asset Value (NAV) calculated internally with the NAV provided by the fund administrator or custodian. This is the most critical check for ensuring the accuracy of the fund's valuation, which is used for pricing fund shares and reporting to investors.

inputs:

- `internal_nav` : The NAV calculated by the internal system.
- `administrator_nav` : The NAV provided by the fund administrator.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM nav_comparison nc
WHERE nc.internal_nav != nc.administrator_nav
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0
- deterministic_or_ai:**
- `should_live_in` : "RULE_ENGINE"
- `notes` : This is a core deterministic rule that requires an exact match.

severity:

- `level` : CRITICAL
- `impact_if_broken` : A mismatch in NAV can lead to incorrect pricing of fund shares, regulatory breaches, and a loss of investor confidence.

example_breaks:

- "Example 1: The internal NAV is \$10.0000, but the administrator's NAV is \$10.0001."
- "Example 2: The internal NAV is \$10.0000, but the administrator's NAV is \$9.9900, a significant difference."

example_fix:

- "The analyst would investigate the source of the discrepancy. This could be due to a pricing error, a missing corporate action, or an incorrect fee accrual. The fix would involve correcting the underlying cause of the break."

- **id:** NAV_002

domain: NAV & Valuation Checks

name: Total Asset Value Match

type: DETERMINISTIC

description: This rule performs an exact match of the total value of the fund's assets as calculated internally against the total asset value provided by the fund administrator. This is a high-level check that helps to identify any significant differences in the valuation of the fund's portfolio.

inputs:

- `internal_total_assets` : The total value of the fund's assets from the internal system.

- `administrator_total_assets` : The total value of the fund's assets from the administrator.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM asset_value_comparison avc  
WHERE avc.internal_total_assets != avc.administrator_total_assets
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A fundamental deterministic rule for high-level asset validation.

severity:

- `level` : HIGH
- `impact_if_broken` : A mismatch in total assets can indicate a significant valuation error or a missing position.

example_breaks:

- "Example 1: The internal total assets are \$100,000,000, but the administrator's total assets are \$100,050,000."
- "Example 2: The internal total assets are \$100,000,000, but the administrator's total assets are \$99,000,000, a significant difference."

example_fix:

- "The analyst would drill down into the individual positions to identify the source of the valuation difference. This could be due to a single security being priced incorrectly or a position being missing from one of the systems."

3.3.2. Tolerance-Based Rules

- **id:** NAV_003

domain: NAV & Valuation Checks

name: NAV Tolerance Match

type: TOLERANCE

description: This rule allows for a match in the NAV even if there is a small difference, provided the difference is within a predefined tolerance. This is often necessary to account for minor rounding differences or slight variations in the calculation methodologies used by the internal system and the fund administrator.

inputs:

- `internal_nav` : The NAV calculated by the internal system.
- `administrator_nav` : The NAV provided by the fund administrator.
- `tolerance_abs` : The absolute tolerance allowed.
- `tolerance_pct` : The percentage tolerance allowed.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM nav_comparison nc
WHERE ABS(nc.internal_nav - nc.administrator_nav) <=
@tolerance_abs
    OR ABS((nc.internal_nav - nc.administrator_nav) /
nc.internal_nav) * 100 <= @tolerance_pct
```

suggested_tolerances:

- `amount_abs` : 0.01
- `amount_pct` : 0.01%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- ``notes**``: A rule-based tolerance for handling minor NAV differences.

severity:

- `level` : MEDIUM

- `impact_if_broken` : A persistent NAV difference, even if small, could indicate a systematic issue that needs to be investigated.

example_breaks:

- "Example 1: The internal NAV is \$10.0000, and the administrator's NAV is \$10.0001, a difference of 1 basis point, which is within the tolerance."
- "Example 2: The internal NAV is \$10.0000, and the administrator's NAV is \$10.0010, a difference of 10 basis points, which is outside the tolerance."

example_fix:

- "If the difference is within tolerance, the analyst may approve the match. If it's a recurring issue, they might investigate the calculation methodologies and work to align them."

3.3.3. Data Quality Rules

- **id:** NAV_004

domain: NAV & Valuation Checks

name: Missing Price

type: DATA_QUALITY

description: This rule flags any security in the portfolio that is missing a price. A missing price makes it impossible to calculate the market value of the position and, therefore, the NAV. This is a critical data quality issue that must be resolved before the NAV can be finalized.

inputs:

- `security_isin` : The ISIN of the security.
- `price` : The price of the security.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM portfolio_positions pp  
WHERE pp.price IS NULL OR pp.price <= 0
```

suggested_tolerances:

- `amount_abs` : N/A

- `amount_pct` : N/A
- `date_offset_days` : N/A
- deterministic_or_ai:**
- `should_live_in` : "RULE_ENGINE"
- `notes**`: A simple but critical data quality check.
- severity:**
- `level` : CRITICAL
- `impact_if_broken` : A missing price will prevent the NAV from being calculated, which can delay client reporting and regulatory filings.
- example_breaks:**
- "Example 1: A new security is added to the portfolio, but the price has not yet been sourced from the market data provider."
- "Example 2: The price for a particular security is showing as zero or a negative value, which is invalid."
- example_fix:**
- "The analyst would need to source the correct price for the security from a reliable market data provider and update the price in the system. They would also check if there are any issues with the data feed."

3.3.4. Edge-Case Rules

- **id:** NAV_005
- domain:** NAV & Valuation Checks
- name:** Stale Price Check
- type:** EDGE_CASE
- description:** This rule flags any security whose price has not been updated for a specified number of days. A stale price can lead to an inaccurate NAV, especially for securities that are actively traded. This rule helps to ensure that the fund's valuation is based on current market prices.
- inputs:**
 - `security_isin` : The ISIN of the security.
 - `price_date` : The date of the last price update.
 - `current_date` : The current business date.

- `stale_price_threshold_days` : The number of days after which a price is considered stale.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM portfolio_positions pp
WHERE DATEDIFF(day, pp.price_date, @current_date) >
@stale_price_threshold_days
```

suggested_tolerances:

- `amount_abs` : N/A
- `amount_pct` : N/A
- `date_offset_days` : 3

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A deterministic rule based on a date threshold.

severity:

- `level` : HIGH
- `impact_if_broken` : A stale price can lead to an inaccurate NAV and a misrepresentation of the fund's value.

example_breaks:

- "Example 1: The price for a security was last updated three business days ago, which is outside the allowed threshold."
- "Example 2: The price for a security has not been updated since a corporate action occurred, leading to an incorrect valuation."

example_fix:

- "The analyst would need to source a current price for the security and update the price in the system. They would also investigate why the price was not updated automatically."

3.4. Fees & Charges Reconciliation Rules

3.4.1. Deterministic Rules

- **id:** FEE_001

domain: Fees & Charges

name: Management Fee Rate Match

type: DETERMINISTIC

description: This rule compares the management fee rate applied by the fund administrator with the rate specified in the fund's prospectus or investment management agreement. This is a critical control to ensure that the fund is being charged the correct amount for investment management services.

inputs:

- `applied_fee_rate` : The management fee rate applied by the administrator.
- `contractual_fee_rate` : The management fee rate specified in the contract.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM fee_calculation fc  
WHERE fc.applied_fee_rate != fc.contractual_fee_rate
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A fundamental deterministic rule for fee verification.

severity:

- `level` : HIGH
- `impact_if_broken` : An incorrect fee rate can lead to overcharging or undercharging of fees, which directly impacts the fund's NAV and the investment manager's revenue.

example_breaks:

- "Example 1: The contractual management fee rate is 1.00%, but the administrator has applied a rate of 1.05%."
- "Example 2: The contractual management fee rate is 1.00%, but the administrator has applied a rate of 0.95%."

example_fix:

- "The analyst would verify the correct fee rate in the fund's documentation and work with the administrator to correct the fee calculation and any resulting overpayment or underpayment."

- **id:** FEE_002

domain: Fees & Charges

name: Brokerage Commission Rate Match

type: DETERMINISTIC

description: This rule compares the brokerage commission rate charged on a trade with the rate agreed upon with the broker. This is an important control for managing trading costs and ensuring that the fund is not being overcharged for trade execution.

inputs:

- `trade_commission_rate` : The commission rate applied to the trade.
- `agreed_commission_rate` : The commission rate agreed with the broker.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM trade_details td  
WHERE td.trade_commission_rate != td.agreed_commission_rate
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct**`: 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"

- ``notes**``: A deterministic rule for verifying trade execution costs.

severity:

- `level` : MEDIUM
- `impact_if_broken` : An incorrect commission rate can lead to higher trading costs, which directly impacts the fund's performance.

example_breaks:

- "Example 1: The agreed commission rate is \$0.01 per share, but the broker has charged \$0.015 per share."
- "Example 2: The agreed commission rate is 0.1% of the trade value, but the broker has charged 0.15%."

example_fix:

- "The analyst would verify the correct commission rate in the broker agreement and contact the broker to correct the charge and request a credit for any overpayment."

3.4.2. Tolerance-Based Rules

- **id:** FEE_003

domain: Fees & Charges

name: Custody Fee Tolerance Match

type: TOLERANCE

description: This rule allows for a match in the custody fee charged by the custodian with the expected fee, even if there is a small difference, provided the difference is within a predefined tolerance. This is often necessary to account for minor variations in the calculation of custody fees, which can be based on a complex schedule of rates.

inputs:

- `custody_fee_amount` : The custody fee amount charged by the custodian.
- `expected_custody_fee_amount` : The expected custody fee amount based on the fee schedule.
- `tolerance_abs` : The absolute tolerance allowed.

logic_pseudocode_sql: |

sql

📄 复制


```
SELECT *
FROM custody_fees cf
WHERE ABS(cf.custody_fee_amount - cf.expected_custody_fee_amount)
> @tolerance_abs
```

suggested_tolerances:

- amount_abs : 10.00
- amount_pct : 0%
- date_offset_days : 0

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A rule-based tolerance for handling minor custody fee discrepancies.

severity:

- level : MEDIUM
- impact_if_broken : A custody fee discrepancy can lead to incorrect expense reporting and a misstatement of the fund's NAV.

example_breaks:

- "Example 1: The expected custody fee is \$1,000, but the custodian has charged \$1,005, a difference of \$5, which is within the tolerance."
- "Example 2: The expected custody fee is \$1,000, but the custodian has charged \$1,050, a difference of \$50, which is outside the tolerance."

example_fix:

- "If the difference is within tolerance, the analyst may approve the match. If it's a larger difference, they would investigate the fee calculation and contact the custodian for clarification."

3.4.3. Data Quality Rules

- id: FEE_004

domain: Fees & Charges

name: Missing Fee Accrual

type: DATA_QUALITY

description: This rule flags any period where a fee has been charged by a service

provider but has not been accrued in the internal books. This is a critical data quality issue, as it can lead to an understatement of the fund's liabilities and an overstatement of its NAV.

inputs:

- `fee_type` : The type of fee (e.g., 'Management Fee', 'Custody Fee').
- `fee_accrual_date` : The date the fee was accrued in the internal system.
- `fee_invoice_date` : The date the fee was invoiced by the service provider.

logic_pseudocode_sql: |

sql

复制

```
SELECT *
FROM fee_invoices fi
LEFT JOIN fee_accruals fa
  ON fi.fee_type = fa.fee_type
 AND fi.fee_invoice_date = fa.fee_accrual_date
WHERE fa.fee_accrual_date IS NULL
```

suggested_tolerances:

- `amount_abs` : N/A
- `amount_pct` : N/A
- `date_offset_days` : N/A

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A data quality check to ensure that all fees are properly accrued.

severity:

- `level` : HIGH
- `impact_if_broken` : A missing fee accrual can lead to an incorrect NAV and a misstatement of the fund's financial position.

example_breaks:

- "Example 1: The custodian has invoiced a monthly custody fee, but the corresponding accrual has not been made in the internal accounting system."

- "Example 2: A performance fee has been calculated by the administrator, but the accrual has not been booked in the general ledger."

example_fix:

- "The analyst would book the missing fee accrual in the internal system and ensure that the fee is properly reflected in the NAV calculation."

3.4.4. Edge-Case Rules

- id: FEE_005

domain: Fees & Charges

name: Performance Fee Waterfall Calculation

type: EDGE_CASE

description: This rule is designed to verify the calculation of performance fees, which are often based on a complex "waterfall" structure. The rule will compare the performance fee calculated by the administrator with the expected fee based on the fund's performance and the terms of the investment management agreement. This is a complex calculation that requires careful verification.

inputs:

- `fund_performance` : The performance of the fund over the calculation period.
- `hurdle_rate` : The hurdle rate specified in the agreement.
- `high_water_mark` : The high water mark for the fund.
- `performance_fee_rate` : The performance fee rate.
- `calculated_performance_fee` : The performance fee calculated by the administrator.

logic_pseudocode_sql: |

Python

📄 复制

```
# Pseudocode for logic
expected_fee = 0
if fund_performance > hurdle_rate:
    excess_return = fund_performance - hurdle_rate
    if fund_nav > high_water_mark:
        expected_fee = excess_return * performance_fee_rate
if abs(expected_fee - calculated_performance_fee) > tolerance:
    flag_break()
```

suggested_tolerances:

- `amount_abs` : 100.00
- `amount_pct` : 0.1%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : The logic is deterministic but requires a complex calculation.

severity:

- `level` : HIGH
- `impact_if_broken` : An incorrect performance fee calculation can have a significant financial impact on both the investment manager and the investors.

example_breaks:

- "Example 1: The calculated performance fee is \$100,000, but the expected fee based on the waterfall calculation is \$95,000."
- "Example 2: The performance fee calculation does not correctly account for the high water mark, leading to an overpayment to the investment manager."

example_fix:

- "The analyst would perform an independent calculation of the performance fee and work with the administrator to correct any errors in their calculation methodology."

3.5. Corporate Actions Reconciliation Rules

3.5.1. Deterministic Rules

- **id:** CA_001

domain: Corporate Actions Impact

name: Stock Split Quantity Adjustment

type: DETERMINISTIC

description: This rule verifies that the quantity of shares held has been correctly adjusted following a stock split. The rule will compare the new quantity of shares in the internal system with the expected quantity based on the split ratio announced by the company.

inputs:

- `original_quantity` : The quantity of shares before the split.
- `split_ratio` : The ratio of the stock split (e.g., 2 for 1).
- `new_quantity` : The quantity of shares after the split.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM corporate_actions ca
JOIN holdings h ON ca.isin = h.isin
WHERE h.new_quantity != (h.original_quantity * ca.split_ratio)
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A deterministic rule based on a mathematical calculation.

severity:

- `level` : CRITICAL
- ``impact_if_broken``: An incorrect quantity adjustment can lead to significant errors in position and NAV reporting.

example_breaks:

- "Example 1: A 2-for-1 stock split is announced, but the internal system only increases the quantity of shares by 50% instead of 100%."
- "Example 2: A reverse stock split is announced, but the internal system does not correctly reduce the quantity of shares."

example_fix:

- "The analyst would manually adjust the quantity of shares in the internal system to reflect the correct split ratio and investigate why the automated adjustment failed."

- **id:** CA_002

domain: Corporate Actions Impact

name: Cash Dividend Amount Match

type: DETERMINISTIC

description: This rule verifies that the amount of a cash dividend received matches the expected amount based on the dividend rate announced by the company and the number of shares held on the record date. The rule will also account for any withholding taxes that may have been deducted.

inputs:

- `dividend_rate` : The dividend rate per share.
- `shares_on_record_date` : The number of shares held on the record date.
- `withholding_tax_rate` : The rate of withholding tax.
- `dividend_amount_received` : The amount of the dividend received.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM dividend_events de
WHERE de.dividend_amount_received != ((de.dividend_rate *
de.shares_on_record_date) * (1 - de.withholding_tax_rate))
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- ``notes**``: A deterministic rule based on a mathematical calculation.

severity:

- `level` : HIGH

- `impact_if_broken` : An incorrect dividend amount can lead to an error in the fund's income and NAV.

example_breaks:

- "Example 1: A dividend of \$0.50 per share is announced, but the fund receives an amount that is less than expected due to an incorrect withholding tax rate being applied."
- "Example 2: The dividend is paid in a foreign currency, and the FX conversion rate used is incorrect, leading to a discrepancy in the amount received."

example_fix:

- "The analyst would verify the dividend announcement and the tax treaty details to determine the correct dividend and tax amounts. They would then contact the custodian to resolve any discrepancies."

3.5.2. Tolerance-Based Rules

- **id:** CA_003

domain: Corporate Actions Impact

name: Rights Issue Entitlement Tolerance

type: TOLERANCE

description: This rule is used to verify the entitlement for a rights issue, where shareholders are given the right to purchase additional shares. The rule will compare the number of rights allocated to the fund with the expected number based on the rights ratio and the number of shares held on the record date. A small tolerance may be applied to account for rounding.

inputs:

- `rights_ratio` : The ratio of rights to shares (e.g., 1 right for every 5 shares).
- `shares_on_record_date` : The number of shares held on the record date.
- `rights_allocated` : The number of rights allocated to the fund.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM rights_issues ri
```

```
WHERE ABS(ri.rights_allocated - FLOOR(ri.shares_on_record_date /  
ri.rights_ratio)) > @tolerance
```

suggested_tolerances:

- amount_abs : 1
- amount_pct : 0%
- date_offset_days : 0

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A rule-based tolerance for handling minor rounding differences in rights allocations.

severity:

- level : MEDIUM
- impact_if_broken : An incorrect rights entitlement can lead to a missed investment opportunity or an incorrect position.

example_breaks:

- "Example 1: A rights issue of 1 for 5 is announced, and the fund holds 1,000 shares, but it is allocated 199 rights instead of the expected 200."
- "Example 2: The fund is not allocated any rights for a security it holds, which is entitled to the rights issue."

example_fix:

- "The analyst would verify the rights issue announcement and the fund's holdings on the record date. They would then contact the custodian to correct the rights allocation."

3.5.3. Data Quality Rules

- id: CA_004

domain: Corporate Actions Impact

name: Missing Corporate Action Event

type: DATA_QUALITY

description: This rule flags any security in the portfolio for which a corporate action event has been announced but has not been recorded in the internal system. A

missing corporate action event can lead to an incorrect valuation and position, as the impact of the event will not be reflected in the fund's books.

inputs:

- `security_isin` : The ISIN of the security.
- `event_type` : The type of corporate action (e.g., 'SPLIT', 'DIVIDEND').
- `event_status` : The status of the event in the internal system.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM corporate_action_announcements caa
LEFT JOIN internal_corporate_actions ica
  ON caa.security_isin = ica.security_isin
  AND caa.event_type = ica.event_type
WHERE ica.event_status IS NULL
```

suggested_tolerances:

- `amount_abs` : N/A
- `amount_pct` : N/A
- `date_offset_days` : N/A

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A data quality check to ensure that all corporate action events are captured.

severity:

- `level` : HIGH
- `impact_if_broken` : A missing corporate action event can lead to a significant error in the fund's valuation and position.

example_breaks:

- "Example 1: A stock split is announced for a security in the portfolio, but the event is not recorded in the internal system."

- "Example 2: A dividend is announced, but the expected income is not accrued in the internal books."

example_fix:

- "The analyst would record the missing corporate action event in the internal system and ensure that the necessary adjustments to positions and cash are made."

3.5.4. Edge-Case Rules

- **id:** CA_005

domain: Corporate Actions Impact

name: Merger Exchange Ratio Verification

type: EDGE_CASE

description: This rule is designed to verify the exchange ratio in a merger or acquisition, where shares of one company are exchanged for shares of another. The rule will compare the number of new shares received with the expected number based on the announced exchange ratio and the number of old shares held.

inputs:

- `old_shares_held` : The number of shares of the old company held.
- `exchange_ratio` : The ratio of new shares to old shares.
- `new_shares_received` : The number of shares of the new company received.

logic_pseudocode_sql: |

Python

📄 复制

```
# Pseudocode for logic
expected_new_shares = old_shares_held * exchange_ratio
if abs(new_shares_received - expected_new_shares) > tolerance:
    flag_break()
```

suggested_tolerances:

- `amount_abs` : 1
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes`: The logic is deterministic but requires a complex calculation.
severity:
- `level` : HIGH
- `impact_if_broken` : An incorrect exchange ratio can lead to a significant error in the fund's position.
example_breaks:
- "Example 1: A merger with an exchange ratio of 0.5 is announced, and the fund holds 1,000 shares of the old company, but it receives 495 shares of the new company instead of the expected 500."
- "Example 2: The fund receives cash in lieu of fractional shares, but the amount is not correctly calculated based on the exchange ratio."
- **example_fix:**
- "The analyst would verify the merger announcement and the exchange ratio. They would then contact the custodian to correct the position and any cash-in-lieu payments."

3.6. FX & Multi-Currency Reconciliation Rules

3.6.1. Deterministic Rules

- **id:** FX_001

domain: FX & Multi-Currency Valuation

name: FX Rate Source Match

type: DETERMINISTIC

description: This rule compares the foreign exchange (FX) rate used for valuation in the internal system with the FX rate used by the custodian or fund administrator. This is a critical check to ensure that the valuation of foreign currency assets and liabilities is consistent across all systems.

inputs:

- `internal_fx_rate` : The FX rate used in the internal system.
- `custodian_fx_rate` : The FX rate used by the custodian.
- `currency_pair` : The currency pair (e.g., 'EURUSD').

logic_pseudocode_sql: |

sql

复制

```
SELECT *  
FROM fx_rate_comparison frc  
WHERE frc.internal_fx_rate != frc.custodian_fx_rate
```

suggested_tolerances:

- amount_abs : 0
- amount_pct : 0%
- date_offset_days : 0

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A fundamental deterministic rule for FX valuation.

severity:

- level : HIGH
- impact_if_broken : A mismatch in FX rates can lead to significant errors in the valuation of foreign assets and the overall fund NAV.

example_breaks:

- "Example 1: The internal system uses a EURUSD rate of 1.1000, but the custodian uses a rate of 1.1005."
- "Example 2: The internal system uses an end-of-day FX rate, while the custodian uses an intraday rate, leading to a difference in valuation."

example_fix:

- "The analyst would verify the correct FX rate from an independent source and work with the custodian to align the rate sources and timing."

• **id:** FX_002

domain: FX & Multi-Currency Valuation

name: Cross-Currency Cash Movement Match

type: DETERMINISTIC

description: This rule is used to reconcile cash movements that involve the exchange of one currency for another. The rule will verify that the amounts of both

currencies involved in the exchange are correct based on the agreed-upon FX rate.

inputs:

- `from_currency` : The currency being sold.
- `from_amount` : The amount of the currency being sold.
- `to_currency` : The currency being bought.
- `to_amount` : The amount of the currency being bought.
- `fx_rate` : The FX rate used for the exchange.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM fx_trades ft  
WHERE ABS((ft.from_amount * ft.fx_rate) - ft.to_amount) >  
@tolerance
```

suggested_tolerances:

- `amount_abs` : 0.01
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A deterministic rule based on a mathematical calculation.

severity:

- `level` : HIGH
- `impact_if_broken` : An incorrect cross-currency cash movement can lead to an error in the fund's cash balances.

example_breaks:

- "Example 1: A trade to sell EUR 1,000,000 and buy USD at a rate of 1.10 results in a credit of USD 1,100,000, but the actual credit is for USD 1,095,000."

- "Example 2: An FX trade is booked with the wrong currency pair, leading to an incorrect exchange of funds."

example_fix:

- "The analyst would verify the details of the FX trade with the counterparty and correct the cash balances in the internal system if necessary."

3.6.2. Tolerance-Based Rules

- id: FX_003

domain: FX & Multi-Currency Valuation

name: FX Rate Tolerance

type: TOLERANCE

description: This rule allows for a match in the FX rate used for valuation even if there is a small difference, provided the difference is within a predefined tolerance. This is often necessary to account for minor variations in the timing of rate fixes or the use of different market data providers.

inputs:

- `internal_fx_rate` : The FX rate used in the internal system.
- `custodian_fx_rate` : The FX rate used by the custodian.
- `tolerance_pct` : The percentage tolerance allowed.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM fx_rate_comparison frc
WHERE ABS((frc.internal_fx_rate - frc.custodian_fx_rate) /
frc.internal_fx_rate) * 100 > @tolerance_pct
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0.1%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"

- `notes` : A rule-based tolerance for handling minor FX rate differences.

severity:

- `level` : MEDIUM
- `impact_if_broken` : A persistent FX rate difference can lead to a significant valuation error for large foreign currency positions.

example_breaks:

- "Example 1: The internal EURUSD rate is 1.1000, and the custodian's rate is 1.1001, a difference of 1 basis point, which is within the tolerance."
- "Example 2: The internal EURUSD rate is 1.1000, and the custodian's rate is 1.1010, a difference of 10 basis points, which is outside the tolerance."

example_fix:

- "If the difference is within tolerance, the analyst may approve the match. If it's a larger difference, they would investigate the rate sources and timing."

3.6.3. Data Quality Rules

- **id:** FX_004

domain: FX & Multi-Currency Valuation

name: Invalid Currency Pair

type: DATA_QUALITY

description: This rule flags any FX transaction or valuation that involves an invalid or unrecognized currency pair. This is a critical data quality check, as an incorrect currency pair can lead to significant errors in FX conversion and valuation.

inputs:

- `from_currency` : The currency being sold.
- `to_currency` : The currency being bought.

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *
FROM fx_trades ft
WHERE ft.from_currency NOT IN (SELECT currency_code FROM
valid_currencies)
```

```
OR ft.to_currency NOT IN (SELECT currency_code FROM
valid_currencies)
```

suggested_tolerances:

- amount_abs : N/A
- amount_pct : N/A
- date_offset_days : N/A

deterministic_or_ai:

- should_live_in : "RULE_ENGINE"
- notes : A simple data quality check against a reference table.

severity:

- level : HIGH
- impact_if_broken : An invalid currency pair will prevent the FX conversion from being performed correctly.

example_breaks:

- "Example 1: An FX trade is booked with the currency pair 'USDEUP' instead of 'EURUSD'."
- "Example 2: A currency pair is entered with the currencies in the wrong order, such as 'USDEUR' instead of 'EURUSD'."

example_fix:

- "The analyst would correct the currency pair in the source data and re-run the reconciliation. They would also work with the data provider to prevent such errors in the future."

3.6.4. Edge-Case Rules

- id: FX_005

domain: FX & Multi-Currency Valuation

name: FX Rounding Difference

type: EDGE_CASE

description: This rule is designed to identify and explain small differences in FX-converted amounts that are due to rounding. When converting between currencies, rounding is often necessary, and small differences can accumulate, especially for

large volumes of transactions. This rule will help to distinguish these rounding differences from genuine errors.

inputs:

- `original_amount` : The original amount in the local currency.
- `fx_rate` : The FX rate used for conversion.
- `converted_amount` : The amount in the base currency after conversion.

logic_pseudocode_sql: |

Python

📄 复制

```
# Pseudocode for logic
expected_converted_amount = original_amount * fx_rate
rounding_difference = abs(converted_amount -
expected_converted_amount)
if rounding_difference <= rounding_tolerance:
    flag_as_rounding_difference()
```

suggested_tolerances:

- `amount_abs` : 0.01
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : The logic is deterministic but requires a specific calculation for rounding.

severity:

- `level` : LOW
- `impact_if_broken` : Rounding differences are typically immaterial and do not represent a genuine error.

example_breaks:

- "Example 1: Converting EUR 100 at a rate of 1.1234 results in an expected USD amount of \$112.34, but the system rounds this to \$112.33."

- "Example 2: A large number of small FX transactions result in a cumulative rounding difference that is flagged by the reconciliation system."

example_fix:

- "The analyst would verify that the difference is due to rounding and would not take any further action. The reconciliation system should be configured to automatically clear these types of breaks."

3.7. Suspense & Exceptions Reconciliation Rules

3.7.1. Deterministic Rules

- **id:** SUSP_001

domain: Suspense & Break Management

name: Unallocated Cash Identification

type: DETERMINISTIC

description: This rule identifies cash entries on the bank or custodian statement that cannot be matched to any trade, fee, or corporate action. This unallocated cash is moved to a suspense account until it can be properly identified and allocated.

inputs:

- `cash_reference` : The reference number of the cash entry.
- `cash_amount` : The amount of the cash entry.
- `matched_trade_id` : The ID of the matched trade (if any).

logic_pseudocode_sql: |

sql

📄 复制

```
SELECT *  
FROM cash_entries ce  
WHERE ce.matched_trade_id IS NULL
```

suggested_tolerances:

- `amount_abs` : 0
- `amount_pct` : 0%
- `date_offset_days` : 0

deterministic_or_ai:

- `should_live_in` : "RULE_ENGINE"
- `notes` : A simple deterministic rule to identify unmatched cash.

**